

Problem Set 3

Kyle Maher

2025-10-10

```
library(fpp3)
set.seed(280) # Reproducibility
```

Question 1

```
y <- aus_production %>%
  select(Beer)
```

Part A

```
ses_forecast <- function(y, alpha, level) {
  n <- length(y)
  l <- numeric(n)
  l[1] <- level

  for (t in 2:n) {
    l[t] <- alpha * y[t] + (1 - alpha) * l[t-1]
  }

  return (l[n])
}

ses_forecast(aus_production$Beer, .15, aus_production$Beer[1])
```

```
[1] 421.2716
```

```
fit <- aus_production %>%
  model(ETS(Beer ~ error("A") + trend("N") + season("N")))
forecast(fit, h = 1)$mean
```

```
[1] 421.3169
```

They are very close. If I could specify a specific value for alpha when calling ETS() I would expect them to be the same.

Part B

```
ses_forecast_sse <- function(y, par) {  
  alpha <- par[1]  
  level <- par[2]  
  
  n <- length(y)  
  l <- numeric(n)  
  l[1] <- level  
  
  for (t in 2:n) {  
    l[t] <- alpha * y[t] + (1 - alpha) * l[t-1]  
  }  
  
  sse <- sum((y - l)^2)  
  
  return (sse)  
}  
  
ses_forecast_sse(  
  aus_production$Beer,  
  c(.15, aus_production$Beer[1])  
)
```

```
[1] 405566.3
```

```
init <- c(alpha = 0.5, level = aus_production$Beer[1])  
  
opt <- optim(init, ses_forecast_sse, y = aus_production$Beer)  
  
opt$par
```

```
      alpha      level  
0.9999994 283.9685747
```

```
fit <- aus_production %>%  
  model(ETS(Beer ~ error("A") + trend("N") + season("N")))  
report(fit)
```

```
Series: Beer  
Model: ETS(A,N,N)  
Smoothing parameters:  
  alpha = 0.1488153
```

```
Initial states:  
  l[0]  
259.6135
```

```
sigma^2: 2588.814
```

	AIC	AICc	BIC
	2891.063	2891.175	2901.216

It is possible that my optimization is not working correctly as my values are $\alpha = 1.000$, and $\text{level} = 283.97$, while the `ETS()` optimization gives $\alpha = .1488$ and $\text{level} = 259.61$.

Part C

```
ses_forecast_optimal <- function(y, par) {  
  init <- c(alpha = par[1], level = par[2])  
  opt <- optim(init, ses_forecast_sse, y = aus_production$Beer)  
  ses_forecast(y, opt$par[1], opt$par[2])  
}  
  
ses_forecast_optimal(aus_production$Beer, c(.5, aus_production$Beer[1]))
```

```
[1] 374
```

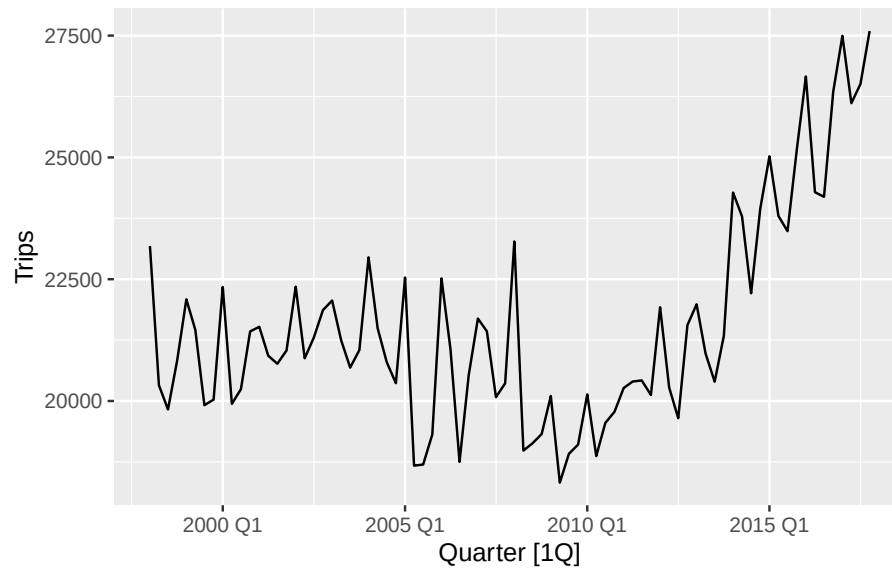
My function still doesn't give the same value as `ETS()`. I'm getting 374 while `ETS()` gives 421.32.

Question 2

```
tourism_sum <- tourism %>%  
  summarise(Trips = sum(Trips))
```

Part A

```
tourism_sum %>%  
  autoplot(Trips)
```



There is consistently one peak per year showing a seasonal pattern. The trend is flat until 2005 where there is a slight decreasing trend until 2010, then there is a strong increasing trend from 2010 onwards.

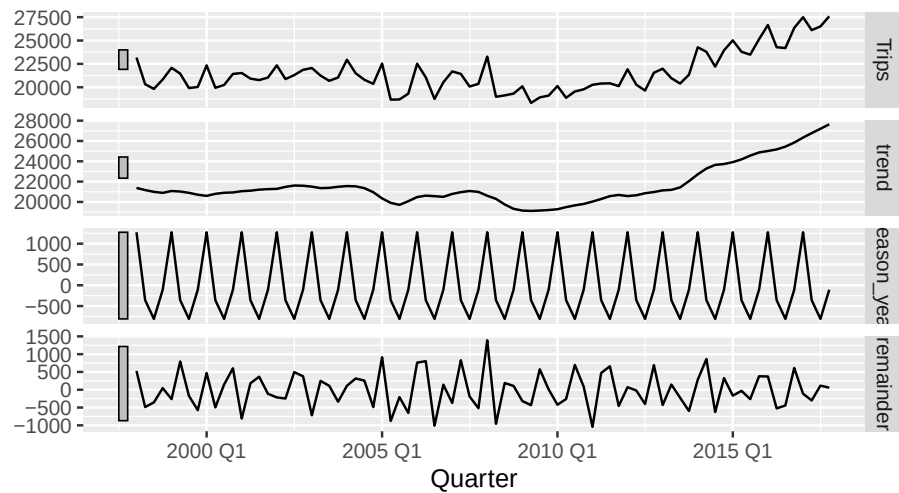
Part B

```
tourism_sum_stl <- tourism_sum %>%
  model(stl = STL(Trips ~ season(window = "periodic")))

components <- components(tourism_sum_stl)
components %>%
  autoplot()
```

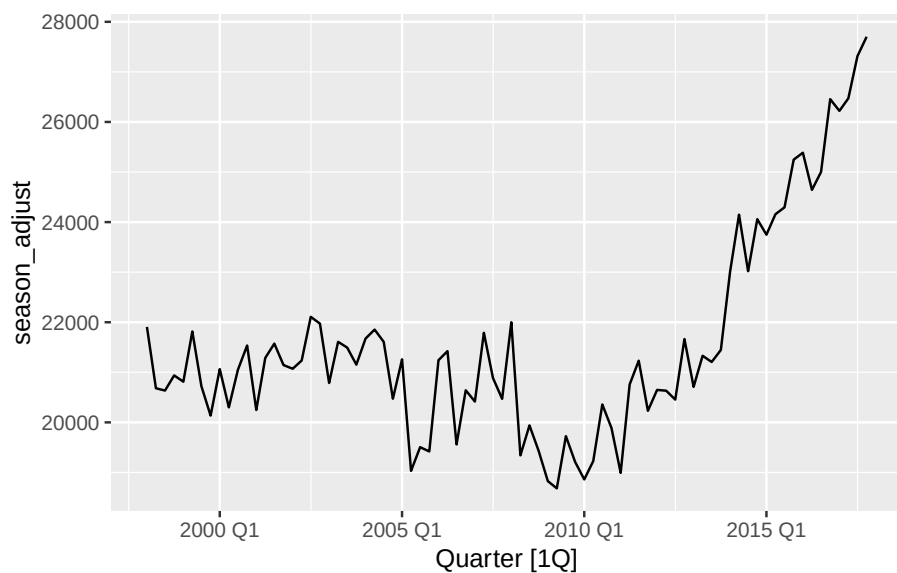
STL decomposition

Trips = trend + season_year + remainder



The trend and season match my initial description.

```
tourism_sum_sa <- components %>%  
  select(season_adjust)  
tourism_sum_sa %>%  
  autoplot(season_adjust)
```



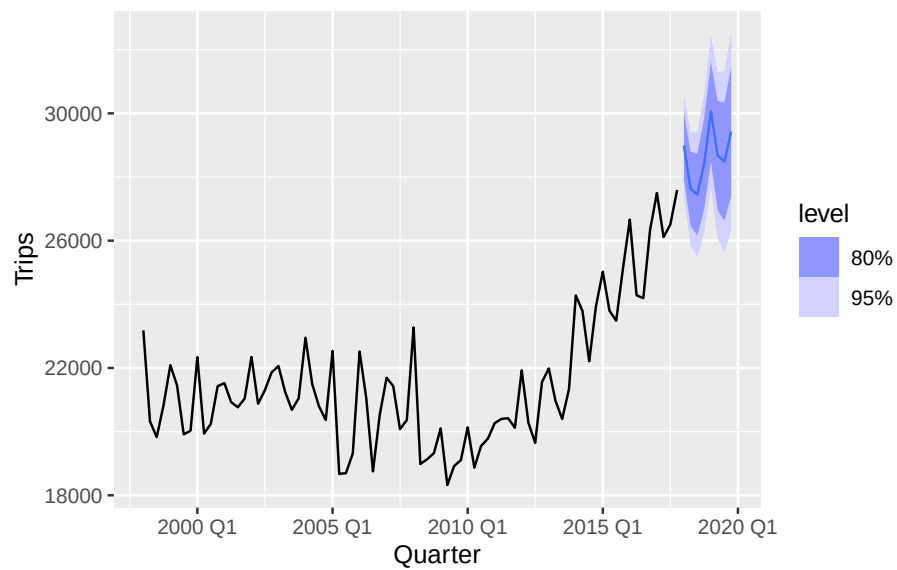
The seasonally adjusted plot includes random variation while showing the trend

clearly.

Part C

```
fit <- tourism_sum %>%
  model(
    decomp_damped = decomposition_model(
      STL(Trips ~ season(window = "periodic")),
      ETS(season_adjust ~ error("A") + trend("Ad") + season("N"))
    )
  )

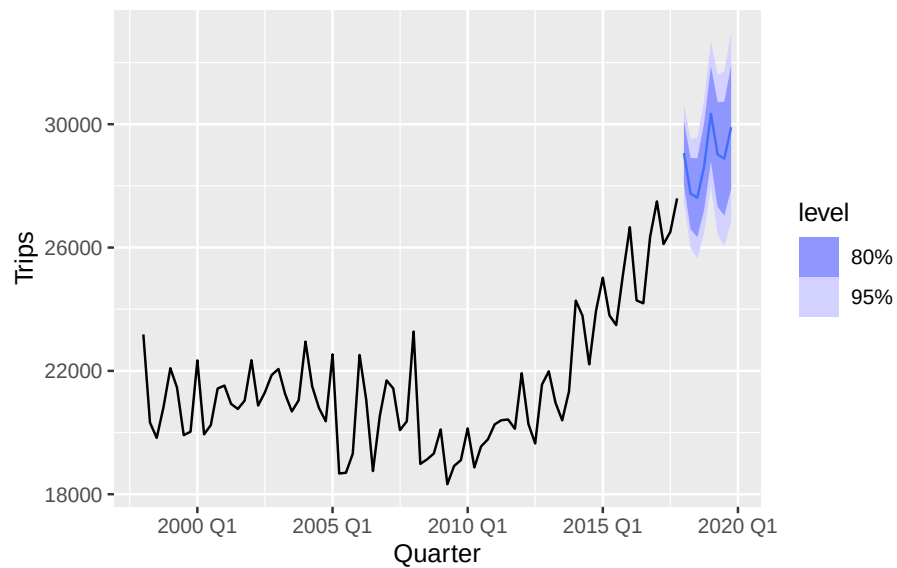
fc <- fit %>%
  forecast(h = 8)
fc %>%
  autoplot(tourism_sum)
```



Part D

```
fit <- tourism_sum %>%
  model(
    decomp = decomposition_model(
      STL(Trips ~ season(window = "periodic")),
      ETS(season_adjust ~ error("A") + trend("A") + season("N"))
    )
  )
```

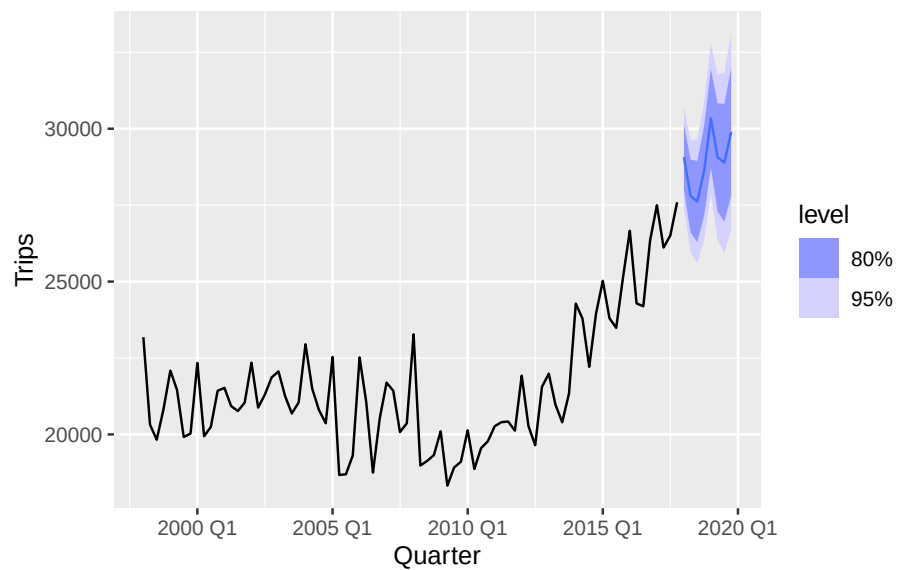
```
fc <- fit %>%
  forecast(h = 8)
fc %>%
  autoplot(tourism_sum)
```



Part E

```
fit <- tourism_sum %>%
  model(
    decomp_AAA = decomposition_model(
      STL(Trips ~ season(window = "periodic")),
      ETS(season_adjust ~ error("A") + trend("A") + season("A"))
    )
  )

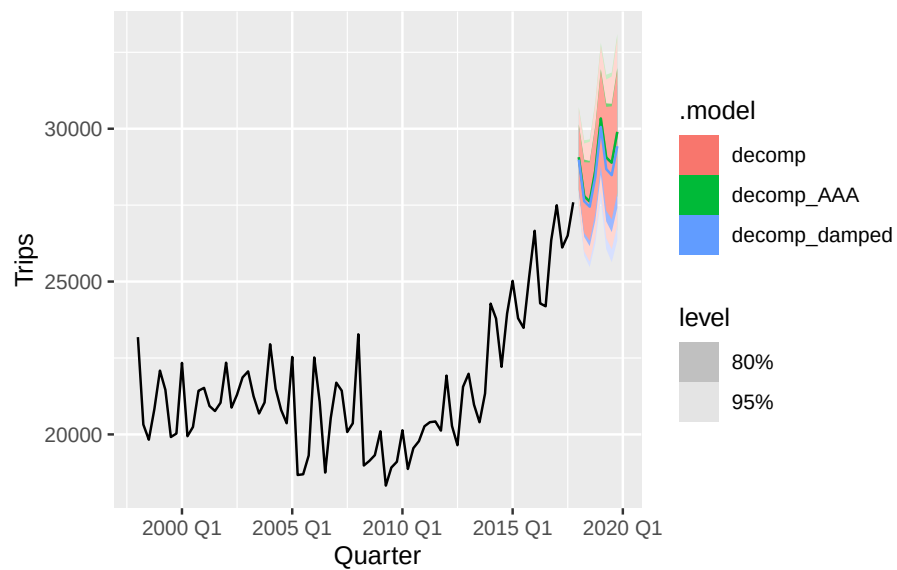
fc <- fit %>%
  forecast(h = 8)
fc %>%
  autoplot(tourism_sum)
```



Part F

```
fit <- tourism_sum %>%
  model(
    decomp_damped = decomposition_model(
      STL(Trips ~ season(window = "periodic")),
      ETS(season_adjust ~ error("A") + trend("Ad") + season("N"))
    ),
    decomp = decomposition_model(
      STL(Trips ~ season(window = "periodic")),
      ETS(season_adjust ~ error("A") + trend("A") + season("N"))
    ),
    decomp_AAA = decomposition_model(
      STL(Trips ~ season(window = "periodic")),
      ETS(season_adjust ~ error("A") + trend("A") + season("A"))
    )
  )

fc <- fit %>%
  forecast(h = 8)
fc %>%
  autoplot(tourism_sum)
```

Part G

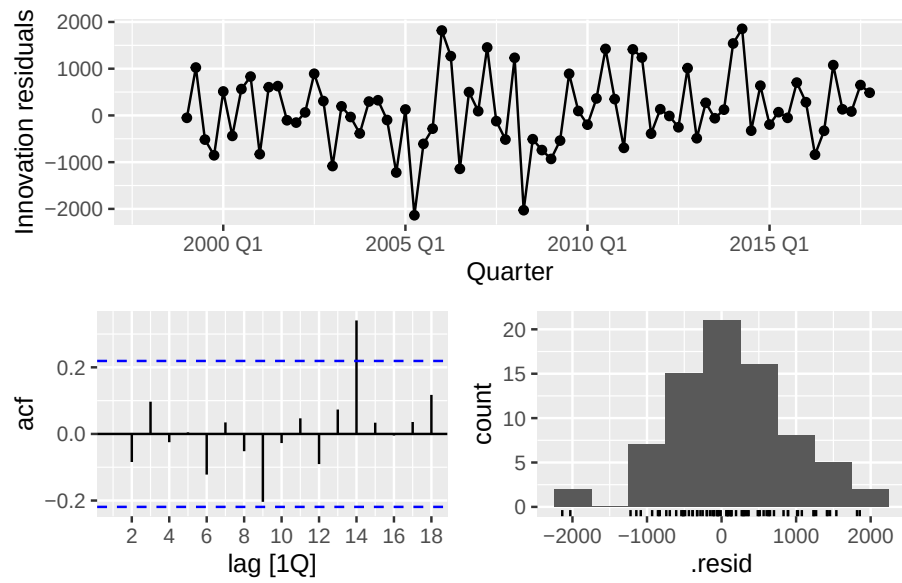
```
accuracy(fit)
```

```
# A tibble: 3 x 10
  .model      .type      ME  RMSE  MAE  MPE  MAPE  MASE  RMSSE  ACF1
  <chr>      <chr>    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 decomp_damped Training  111.  801.  615.  0.393  2.90  0.648  0.660 -
0.00133
2 decomp      Training  113.  801.  611.  0.415  2.89  0.644  0.659 -
0.00186
3 decomp_AAA   Training  126.  802.  610.  0.475  2.88  0.642  0.660 -
0.0119
```

The results are very similar. The dampened decomposition from part C is a good option here due to the fpp3 author's praise of dampend models.

Part H

```
fit %>%
  select(decomp_damped) %>%
  gg_tsresiduals()
```



The residuals appear to be random noise except for significant autocorrelation at lag 14 which is interesting, but not particularly concerning.

③ Show the forecast Variance for $ETS(A,N,N)$ is $\sigma^2[1 + \alpha^2(h-1)]$

$ETS(A,N,N)$ is Simple exponential Smoothing with follows the form $\hat{Y}_{T+1|T} = \alpha Y_T + \alpha(1-\alpha)Y_{T-1} + \alpha(1-\alpha)^2 Y_{T-2} + \dots$

$$V(t_1|t_0) = V(t_0 + \epsilon_1 | t_0)$$

$$= \sigma^2$$

$$V(t_2|t_0) = V(t_1 + \epsilon_2 | t_0)$$

$$= V(t_0 + \alpha t_1 + \epsilon_2 | t_0)$$

$$= \alpha^2 \sigma^2 + \sigma^2$$

$$V(t_3|t_0) = V(t_2 + \epsilon_3 | t_0)$$

$$= V(t_1 + \alpha \epsilon_2 + \epsilon_3 | t_0)$$

$$= V(t_0 + \alpha \epsilon_1 + \alpha \epsilon_2 + \epsilon_3 | t_0)$$

$$= \alpha^2 \sigma^2 + \alpha^2 \sigma^2 + \sigma^2$$

$$= \sigma^2 [1 + 2\alpha^2]$$

$$V(t_4|t_0) = V(t_3 + \epsilon_4 | t_0)$$

$$= V(t_2 + \alpha \epsilon_3 + \epsilon_4 | t_0)$$

$$= V(t_1 + \alpha \epsilon_2 + \alpha \epsilon_3 + \epsilon_4 | t_0)$$

$$= V(t_0 + \alpha \epsilon_1 + \alpha \epsilon_2 + \alpha \epsilon_3 + \epsilon_4 | t_0)$$

$$= \alpha^2 \sigma^2 + \alpha^2 \sigma^2 + \alpha^2 \sigma^2 + \sigma^2$$

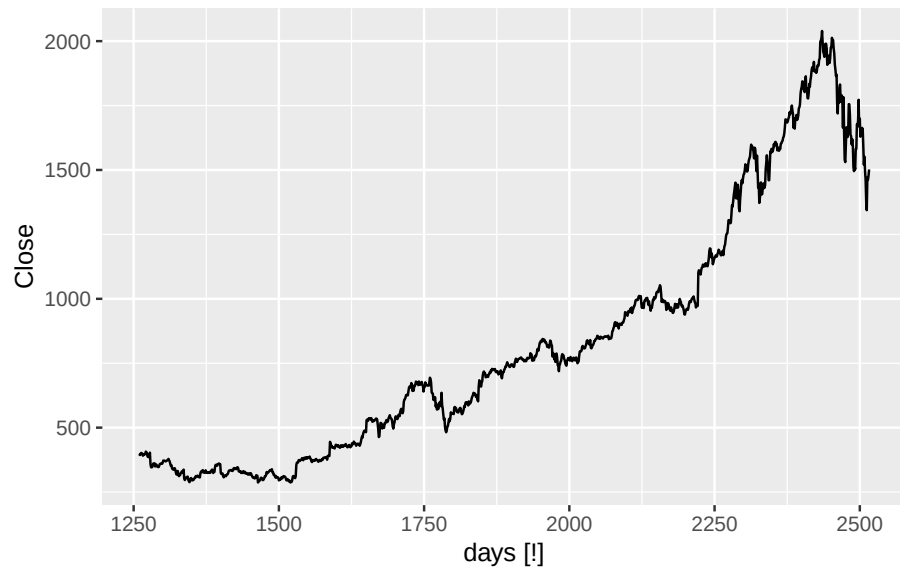
$$= \sigma^2 [1 + 3\alpha^2]$$

⋮

$$V(t_h|t_0) = \sigma^2 [1 + (h-1)\alpha^2]$$

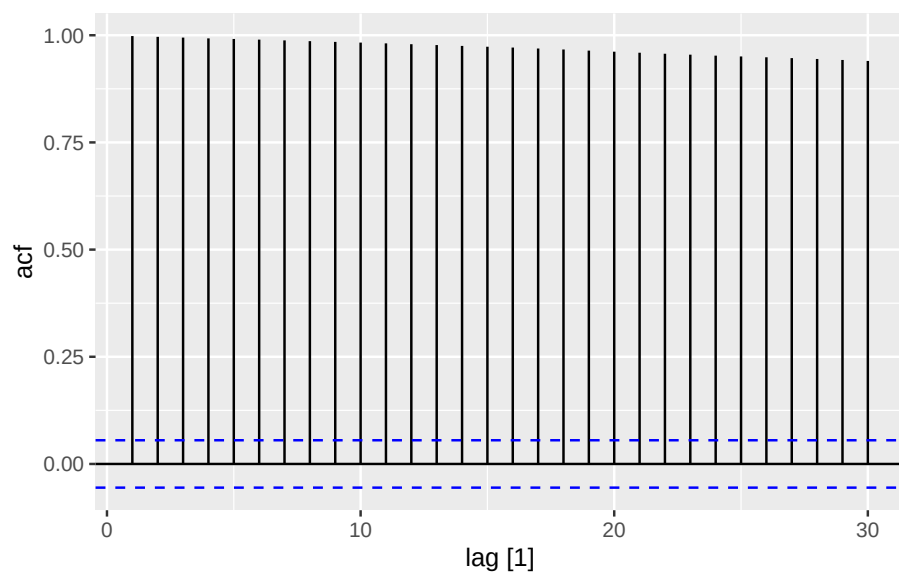
Question 4

```
amazon <- gafa_stock %>%  
  mutate(days = row_number()) %>%  
  update_tsibble(index = days) %>%  
  filter(Symbol == "AMZN") %>%  
  select(Close)  
  
amazon %>%  
  autoplot()
```



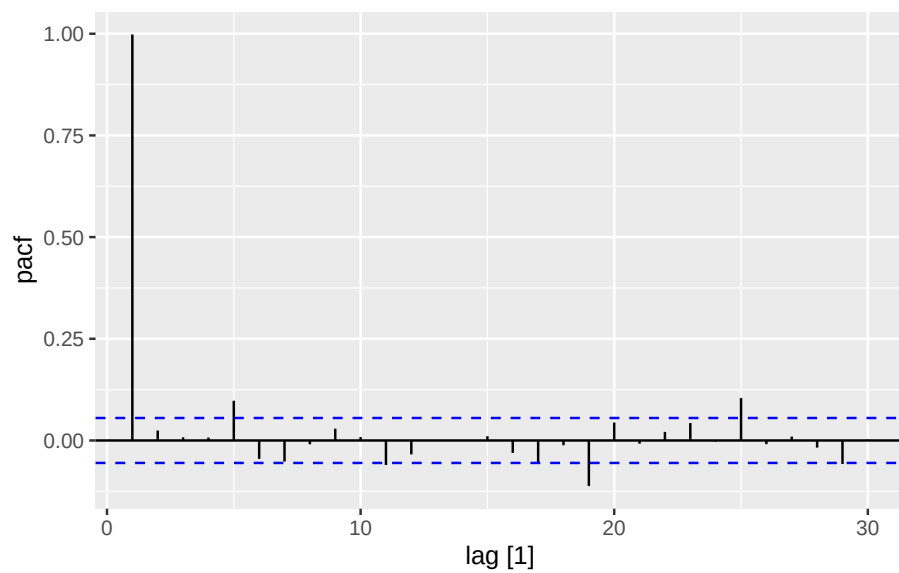
For the basic time series plot an increasing trend is visible for most of the data which indicates it is not stationary. The distribution of closing price depends on the subset of days so the series is not stationary.

```
amazon %>%  
  ACF() %>%  
  autoplot()
```



The Autocorrelation Function output shows extreme autocorrelation between points at all lags shown with a gradual decline in autocorrelation significance. If the series was stationary we would expect no significant autocorrelation here.

```
amazon %>%
  PACF() %>%
  autoplot()
```



The Partial Autocorrelation output shows that most of the autocorrelation is due to adjacent observations but there is still autocorrelation at lags 5, 19, and 25 even when that adjacent autocorrelation is accounted for. Again, a stationary series typically shows no autocorrelation.

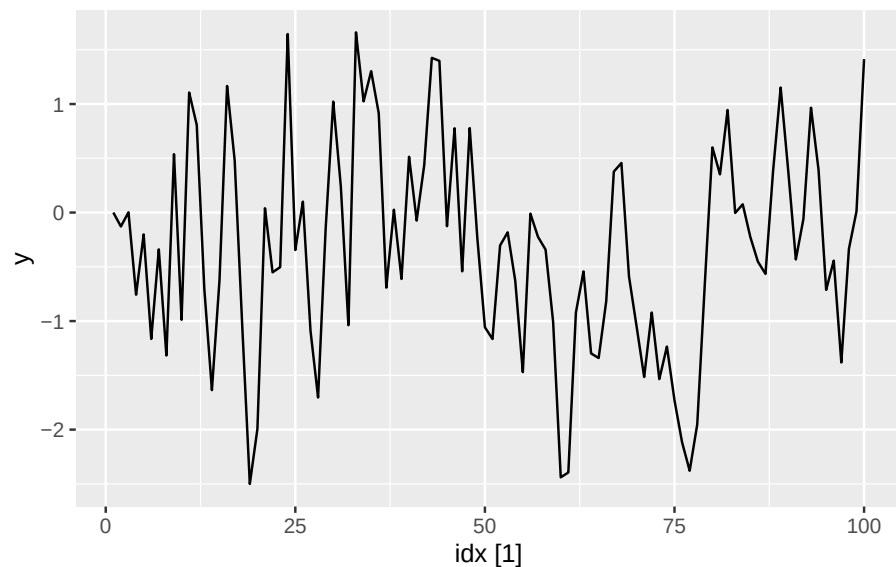
Question 5

Part A

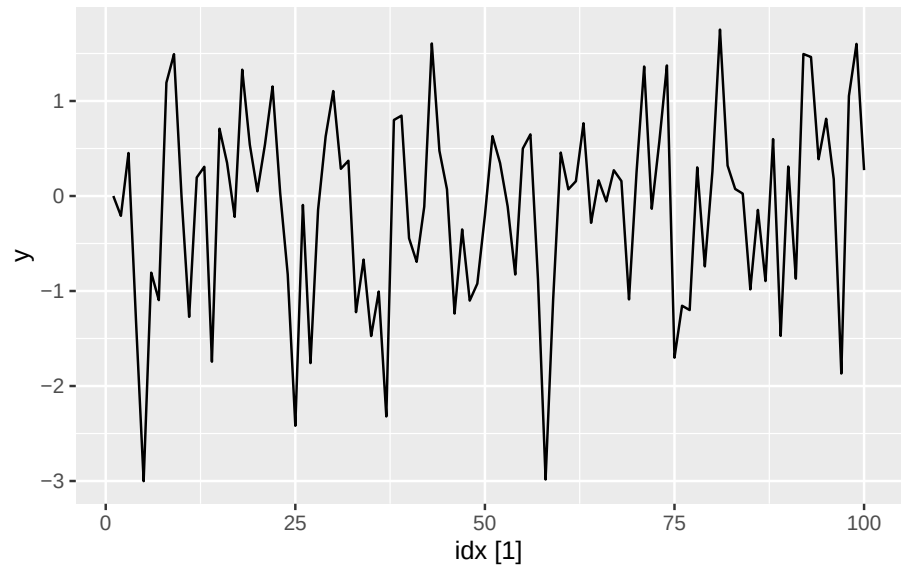
```
simulate_AR <- function(phi) {  
  y <- numeric(100)  
  e <- rnorm(100)  
  for(i in 2:100) {  
    y[i] <- phi*y[i-1] + e[i]  
  }  
  sim <- tsibble(idx = seq_len(100), y = y, index = idx)  
  return (sim)  
}  
sim <- simulate_AR(.6)
```

Part B

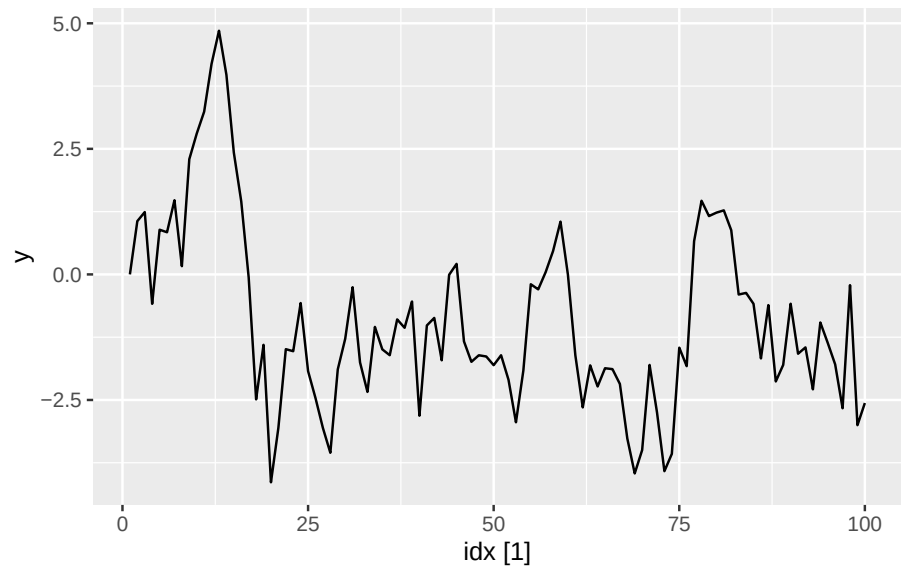
```
sim %>%  
  autoplot(y)
```



```
simulate_AR(.05) %>%  
  autoplot(y)
```



```
simulate_AR(.95) %>%  
  autoplot(y)
```



The parameter ϕ affects how quickly the simulated data changes per period. Low ϕ values result in simulated data that changes rapidly while high ϕ

values result in slower changes. Additionally, larger values of ϕ resulted in a greater range for y .

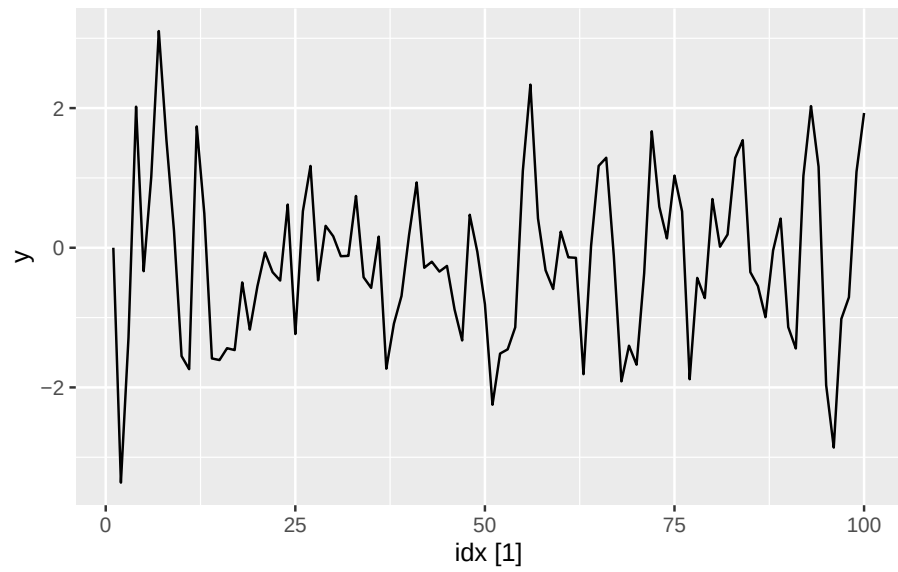
From a statistical standpoint, ϕ is the proportion of the previous observation that influences the next observation. This does align with changes seen in the plots when altering ϕ . When the previous observation has a large effect (when ϕ is close to 1) then the data is less likely to change rapidly and has a more pronounced though still changing trend.

Part C

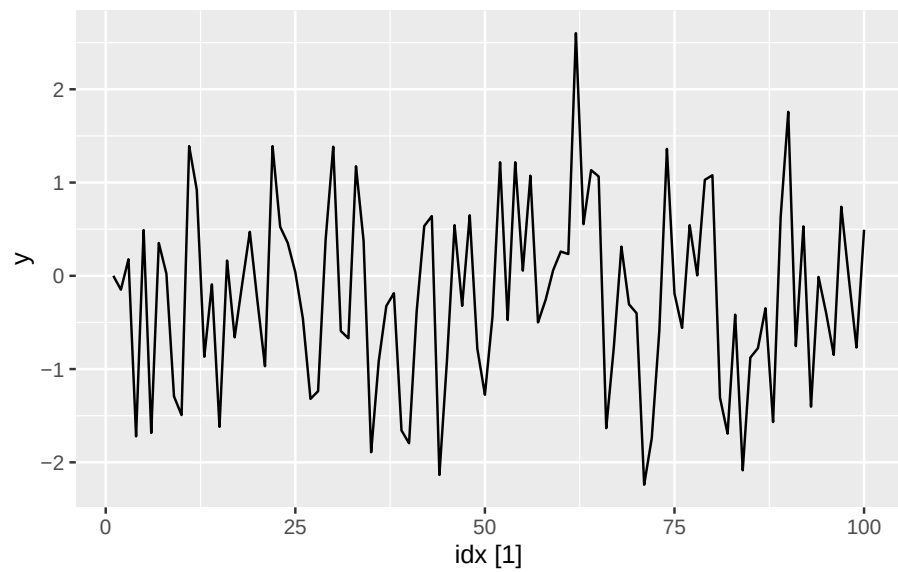
```
simulate_MA <- function(theta) {  
  y <- numeric(100)  
  e <- rnorm(100)  
  
  for (i in 2:100) {  
    y[i] <- e[i] + theta * e[i - 1]  
  }  
  
  sim <- tsibble(idx = seq_len(100), y = y, index = idx)  
  return(sim)  
}
```

Part D

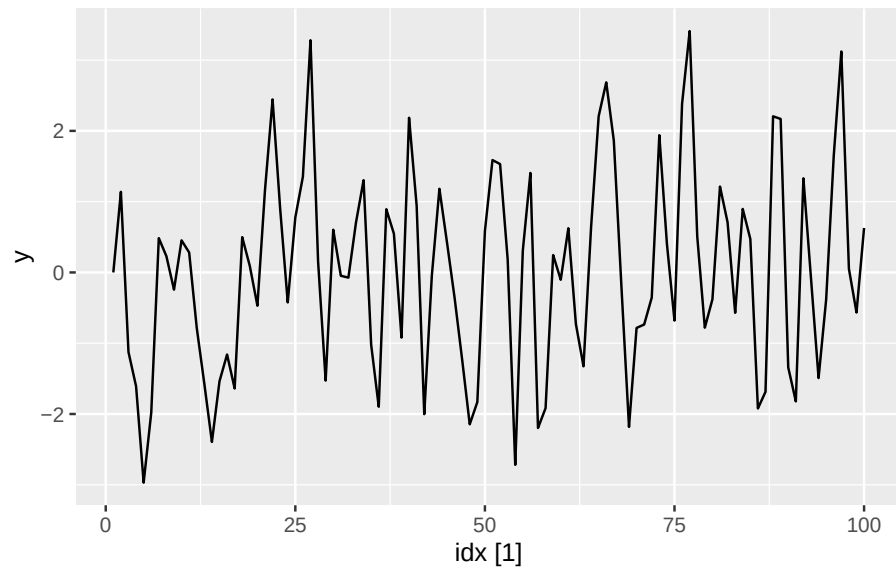
```
simulate_MA(0.6) %>%  
  autoplot(y)
```

```
simulate_MA(.01) %>%  
  autoplot(y)
```



```
simulate_MA(0.99) %>%  
  autoplot(y)
```



Similar to ϕ , the greater θ is the more the previous observation influences the next one. This increase in autocorrelation results in more variation in y with less alterations above and below zero.

From a statistical standpoint this makes sense as ϕ represents the proportion of the error in the previous observation that influences the next observation. A greater value causes adjacent observations to be more correlated.

Part E

```
phi <- .6
theta <- .6

y <- numeric(100)
e <- rnorm(100)
for (i in 2:100) {
  y[i] <- theta * e[i - 1] + phi * y[i - 1] + e[i]
}
arma <- tsibble(idx = seq_len(100), y = y, index = idx)
```

Part F

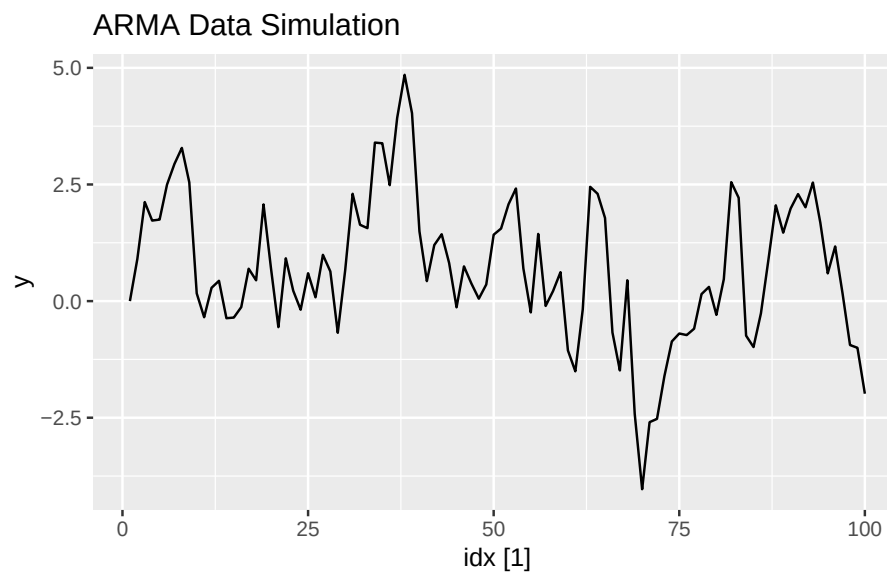
```
phi_1 <- -.8
phi_2 <- .3

y <- numeric(100)
```

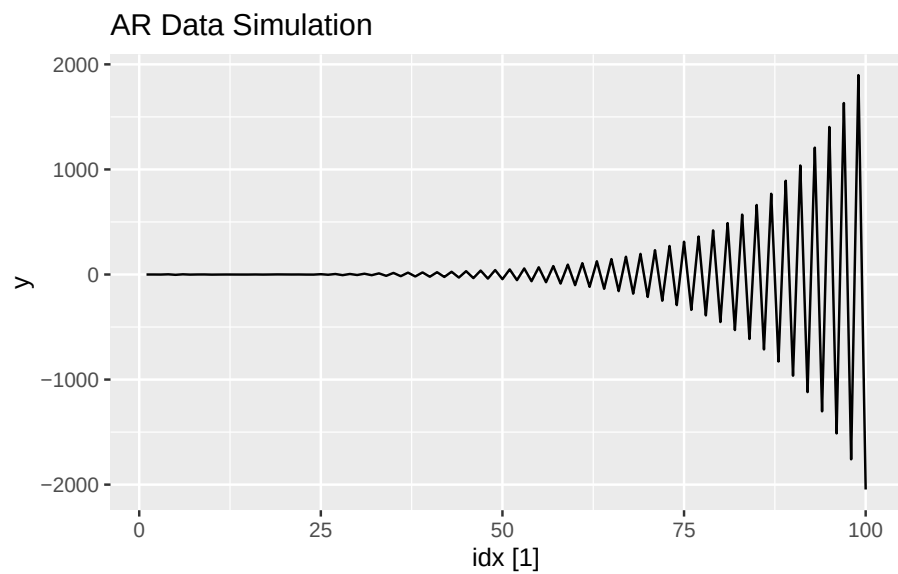
```
e <- rnorm(100)
for (i in 3:100) {
  y[i] <- phi_1 * y[i - 1] + phi_2 * y[i - 2] + e[i]
}
ar <- tsibble(idx = seq_len(100), y = y, index = idx)
```

Part G

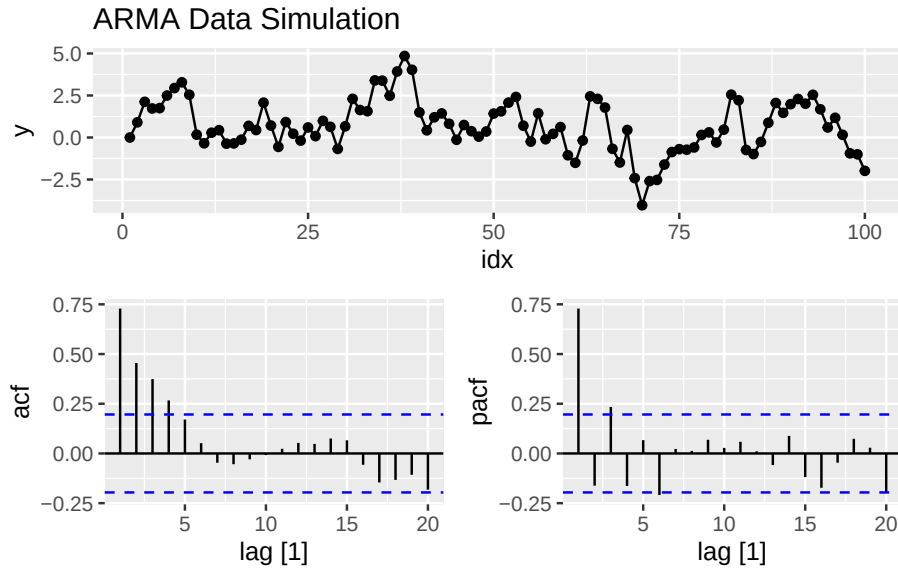
```
arma %>%
  autoplot(y) +
  labs(title = "ARMA Data Simulation")
```



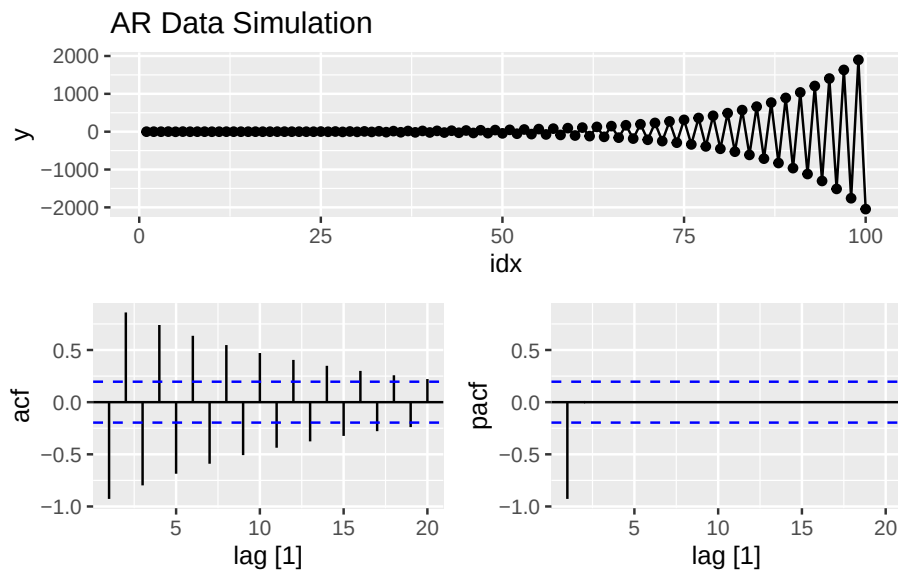
```
ar %>%
  autoplot(y) +
  labs(title = "AR Data Simulation")
```



```
arma %>%
  gg_tsdisplay(y, plot_type = "partial") +
  labs(title = "ARMA Data Simulation")
```



```
ar %>%
  gg_tsdisplay(y, plot_type = "partial") +
  labs(title = "AR Data Simulation")
```



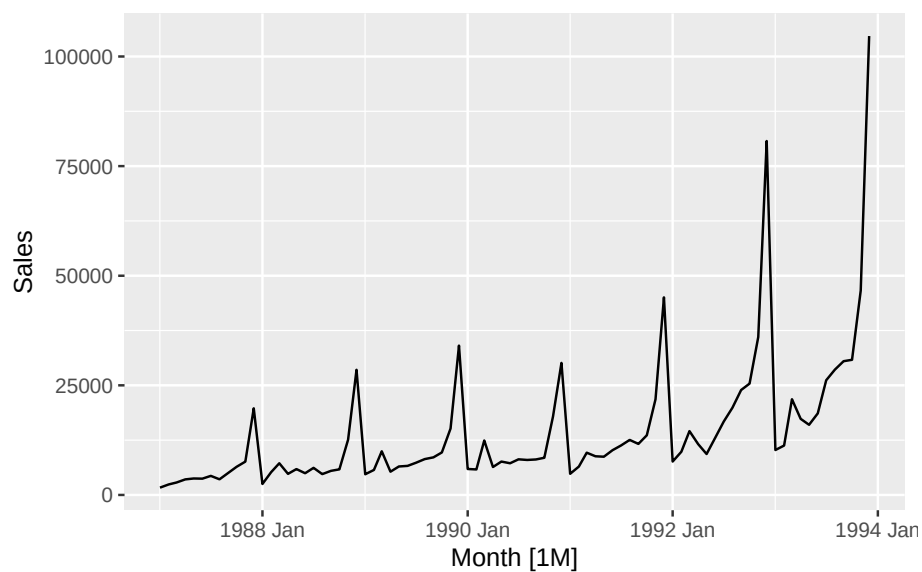
The data generated using the AR(2) model grows in magnitude while oscillating and is not stationary. The partial autocorrelation shows that all autocorrelation can be explained through adjacent observations.

The data generated from the ARMA(1) model also shows significant autocorrelation from the ACF() and PACF() functions.

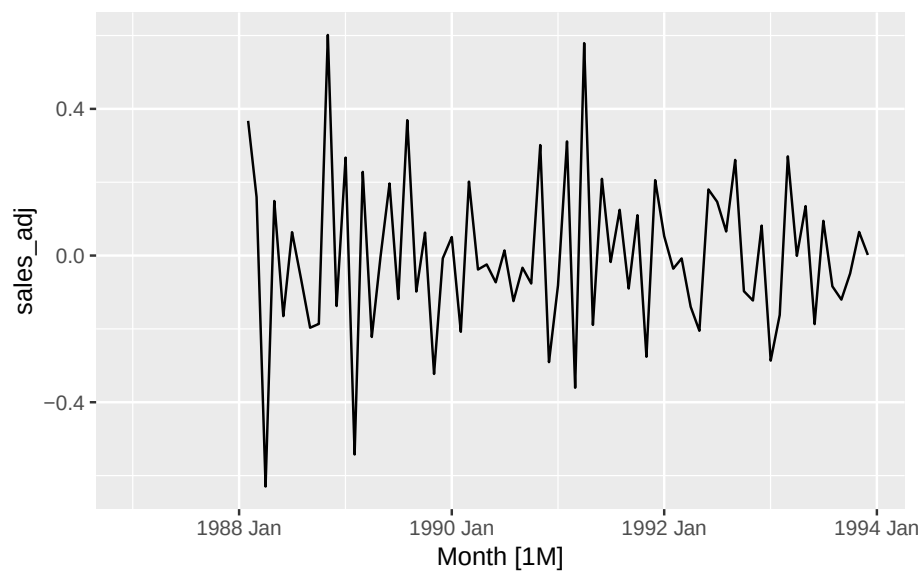
Question 6

Part A

```
souvenirs %>%
  autoplot(Sales)
```



```
souvenirs %>%
  mutate(sales_adj = difference(difference(log(Sales), lag = 12), lag = 1)
) %>%
  autoplot(sales_adj)
```



The log transformation eliminates the heteroskedasticity, the difference of 12 removes seasonality, and then a first difference is needed to fully transform the series to be stationary.

Part B

The twelfth difference followed by the first difference can be written in Backshift notation as $(1 - B)(1 - B^{12})(\log(Sales))$.

⑦ Let $x \in \mathbb{R}^n$ and $y \in \mathbb{R}^m$ be random vectors
and let $A \in \mathbb{R}^{k \times n}$ and $B \in \mathbb{R}^{l \times m}$ be fixed matrices,

⑧ Prove that $\text{Cov}(Ax, By) = A \text{Cov}(x, y) B^T$.

Given the definition of cross-covariance of real random vectors:

$$\text{Cov}(X, Y) = E[(X - E(X))(Y - E(Y))^T],$$

$$\begin{aligned}\text{Cov}(Ax, By) &= E[(Ax - E(Ax))(By - E(By))^T] \\&= E[(Ax - AE(x))(By - BE(y))^T] \\&= E[(A(x - E(x)))(B(y - E(y)))^T] \\&= E[A(x - E(x))((y - E(y))^T B^T)] \\&= A E[(x - E(x))(y - E(y))^T] B^T \\&= \boxed{A \text{Cov}(x, y) B^T}.\end{aligned}$$

⑨ Prove that $\text{Cov}(Ax) = A \text{Cov}(x) A^T$.

Given $\text{Cov}(Ax, By) = A \text{Cov}(x, y) B^T$.

$$\begin{aligned}\text{Cov}(Ax) &= \text{Cov}(Ax, Ax) \\&= \boxed{A \text{Cov}(x) A^T}.\end{aligned}$$

(7c) Suppose $y = X\beta + \varepsilon$ with $\varepsilon \sim N(0, \sigma^2)$.

Given $\hat{\beta} = (X^T X)^{-1} X^T y$, Prove $\text{Cov}(\hat{\beta}) = \sigma^2 (X^T X)^{-1}$.

$$\begin{aligned}\hat{\beta} &= (X^T X)^{-1} X^T (X\beta + \varepsilon) \\ &= (X^T X)^{-1} X^T \beta + (X^T X)^{-1} X^T \varepsilon\end{aligned}$$

$$\begin{aligned}\text{Cov}(\hat{\beta}) &= \text{Cov}((X^T X)^{-1} X^T \beta + (X^T X)^{-1} X^T \varepsilon) \\ &= \text{Cov}((X^T X)^{-1} X^T \varepsilon) \\ &= (X^T X)^{-1} X^T \text{Cov}(\varepsilon) [(X^T X)^{-1} X^T]^T \\ &= (X^T X)^{-1} X^T \sigma^2 [X^T (X^T X)^{-1} X^T] \\ &= \sigma^2 (X^T X)^{-1} (X^T X) (X^T X)^{-1} \\ &= \sigma^2 (X^T X)^{-1} (X^T X) (X^T X)^{-1} \\ &= \sigma^2 (X^T X)^{-1}\end{aligned}$$